

Designing Enterprise Retrieval-Augmented Generation Systems

A Technical Research Report

Author: Hooria Zahoor Track: NLP and AI Agents

Abstract

Retrieval-Augmented Generation (RAG) grounds large language models in an organization's private knowledge without retraining, but production-grade enterprise deployments demand deliberate engineering across the whole pipeline. This report surveys the core design decisions — architecture, embeddings, chunking, retrieval, prompt construction, vector databases, and hallucination reduction — and closes with practical directions for improving accuracy, latency, and trust at scale.

1. Architecture

An enterprise RAG system separates cleanly into an offline ingestion pipeline and an online query pipeline. Ingestion extracts, cleans, chunks, and embeds documents into a vector store with metadata (source, page, access permissions, timestamp). The query pipeline embeds the user's question, retrieves and reranks candidate chunks, assembles context, constructs a grounded prompt, and generates a cited response, optionally passing through a verification layer before it reaches the user.

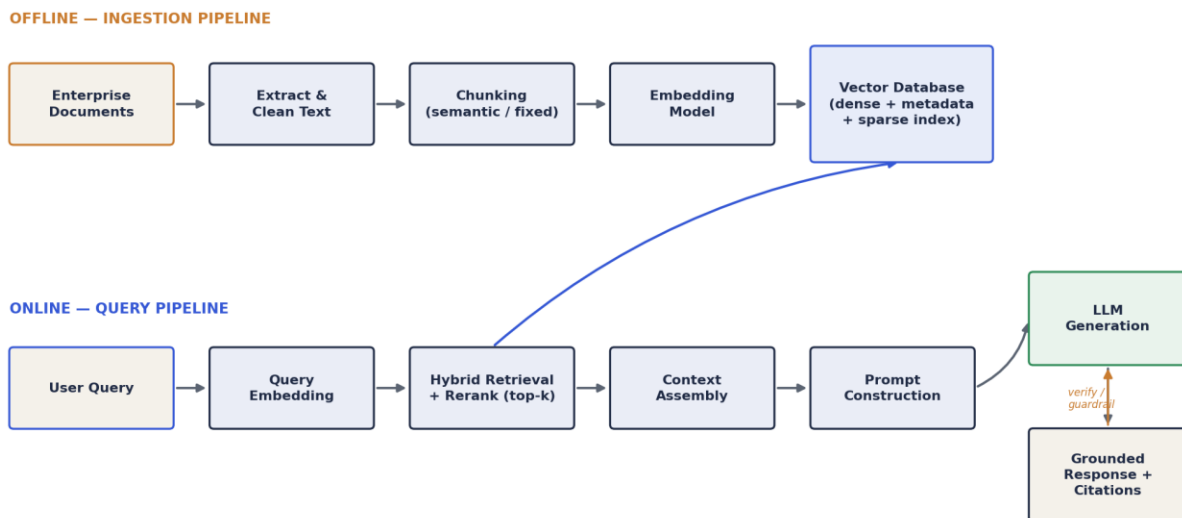


Figure 1. End-to-end enterprise RAG architecture — ingestion (offline) and query (online) pipelines.

- **Modularity:** keeps embedding and generation models decoupled, so either can be upgraded independently;
- **Governance:** access control, versioning, and freshness checks run at the retrieval layer, not inside the model.

Real-world example: Morgan Stanley built a GPT-4-based assistant for its financial advisors that follows exactly this split: tens of thousands of research PDFs are ingested and indexed offline, while an advisor's question runs the online query pipeline in real time, always pointing back to the source report.

2. Embeddings

Embedding models convert text into dense vectors that place semantically similar content close together. Enterprise deployments typically choose between hosted API embeddings (fast to adopt, no infrastructure, but data leaves the perimeter unless a private endpoint is used) and self-hosted open models (full data control, tunable to domain vocabulary, but requiring GPU capacity). Domain-adapted or fine-tuned embeddings materially improve retrieval quality for specialized corpora — legal, medical, or financial text — where general-purpose embeddings under-distinguish jargon-heavy passages.

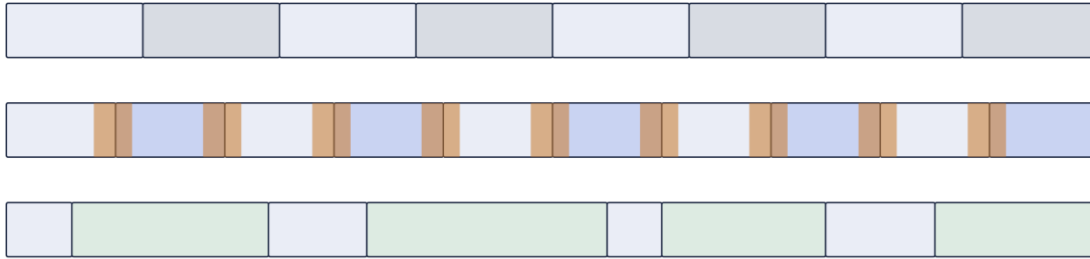
- **Selection criteria:** dimensionality (higher can mean better recall, at higher storage and query cost);
- multilingual support if the corpus spans languages;
- consistency — re-embedding the entire corpus is required whenever the model changes.

Real-world example: A hospital network might use a general-purpose embedding model for scheduling and billing FAQs, but switch to a clinically fine-tuned embedding for its drug-interaction literature — generic embeddings tend to blur precise medical terms that a domain model keeps distinct.

3. Chunking

Chunking controls what unit of text is retrieved and shown to the model, so it directly shapes both recall and answer coherence. Fixed-size chunking is simple and fast but can sever a sentence or table mid-thought; adding overlap (10–20%) mitigates boundary loss at the cost of index size. Semantic or recursive chunking splits at natural paragraph, heading, or section boundaries, preserving meaning at the price of variable chunk length and added preprocessing.

Chunking Strategies



Shaded = paragraph/section-aware chunk

Amber = overlapping token span

Figure 2. Fixed-size, fixed-size-with-overlap, and semantic chunking side by side.

- **Rule of thumb:** 400–800 tokens per chunk is a common enterprise starting point, tuned per corpus.

Real-world example: A law firm indexing a merger agreement chunks by numbered clause so retrieval returns a whole obligation intact, while a support team indexing chat transcripts uses fixed 500-token windows with overlap, since transcripts have no natural headings to split on.

4. Retrieval

Pure dense (embedding) retrieval misses exact-match signals like product codes, error IDs, or names; pure keyword (BM25) search misses paraphrase and synonymy. Hybrid retrieval — combining dense similarity with sparse keyword scoring, then merging or reranking the results — consistently outperforms either method alone in enterprise settings where terminology is precise and queries are conversational. A cross-encoder reranker applied to the top 20–50 hybrid candidates before selecting the final top-k further improves precision, at added latency cost that most enterprise SLAs can absorb.

- **Metadata filtering:** filter search to permitted or relevant document sets before scoring, not after.
- **Top-k tuning:** k is typically 3–8 chunks; too few starves the model of context, too many dilutes attention and raises cost.

Real-world example: GitHub Copilot-style coding assistants combine exact keyword matches on function or variable names with semantic search for natural-language questions like “how do I retry a failed request” — semantic search alone would often miss the one function name the developer actually needs.

5. Prompt Construction

The prompt is where retrieval quality either pays off or is wasted. Effective enterprise prompts number each retrieved chunk with its source and page, instruct the model to answer only from the supplied context, require an explicit “I don't know” when context is insufficient, and request a structured citation list the application can render and verify. Ordering matters: placing the most relevant chunks nearest the question reduces the effect of position bias in long-context models.

- **Stability:** keep system instructions stable and version them, since prompt changes shift behavior as much as model changes do.

Real-world example: *Perplexity.ai's answers show numbered, clickable citations next to each claim — a direct product of a prompt that requires the model to tag every statement with the source number it came from, the same pattern used in Figure 1's response layer.*

6. Vector Databases

The vector store must support approximate nearest-neighbor search at scale, metadata filtering, and incremental updates without full reindexing. Common choices include managed services (Pinecone, Azure AI Search, Vertex AI Search), self-hosted engines (Milvus, Weaviate, Qdrant), and embedded libraries (FAISS, Chroma) suited to smaller or single-tenant deployments. Enterprise selection criteria typically weigh multi-tenancy and row-level security, horizontal scalability, hybrid (dense + sparse) query support, and operational maturity — backup, monitoring, and SLA guarantees — over raw benchmark speed.

Real-world example: *Many SaaS copilots (including early Notion AI features) build on managed services like Pinecone to move fast, while a bank handling regulated customer data is more likely to self-host Milvus or Qdrant inside its own cloud tenant so embeddings never leave its network perimeter.*

7. Hallucination Reduction

RAG reduces but does not eliminate hallucination, since a model can still ignore, misread, or over-generalize from retrieved context. A layered defense is more reliable than any single technique: ground generation strictly in ranked retrieved context, constrain the prompt to require citations and permit abstention, verify that generated claims are traceable to a cited source before the response is shown, and route low-confidence or unverifiable answers to human review.

Hallucination-Reduction Layers



Figure 3. Layered hallucination-reduction pipeline, from retrieval grounding to human review.

Real-world example: *Klarna's customer-service assistant answers strictly from its help-center articles and hands the conversation to a human agent rather than guessing when it can't find a grounded answer — a live production example of the “ground, constrain, and escalate” pattern in Figure 3.*

8. Future Improvements

- **Agentic retrieval:** multi-step retrieval that reformulates or decomposes a query based on intermediate results, for questions no single retrieval pass can answer;
- **Adaptive retrieval:** dynamic top-k and chunk size chosen per query rather than fixed globally;
- **Continuous ingestion:** incremental, event-driven reindexing so answers reflect same-day document changes;
- **Evaluation loops:** retrieval and citation-faithfulness metrics (not just generation quality) tracked in production, with human feedback looped back into reranker and prompt tuning.

Real-world example: *Perplexity's “Pro Search” and similar agentic search tools already issue an initial search, read the results, then fire follow-up searches before composing an answer — a preview of the multi-step, agentic retrieval enterprise RAG systems are moving toward.*